



Ensemble Models

CS 372 Machine Learning I

Dr. Ola Amayri

Summer, 2024

Outline

- Model Ensemble
- Bagging
- Random Forest
- Boosting

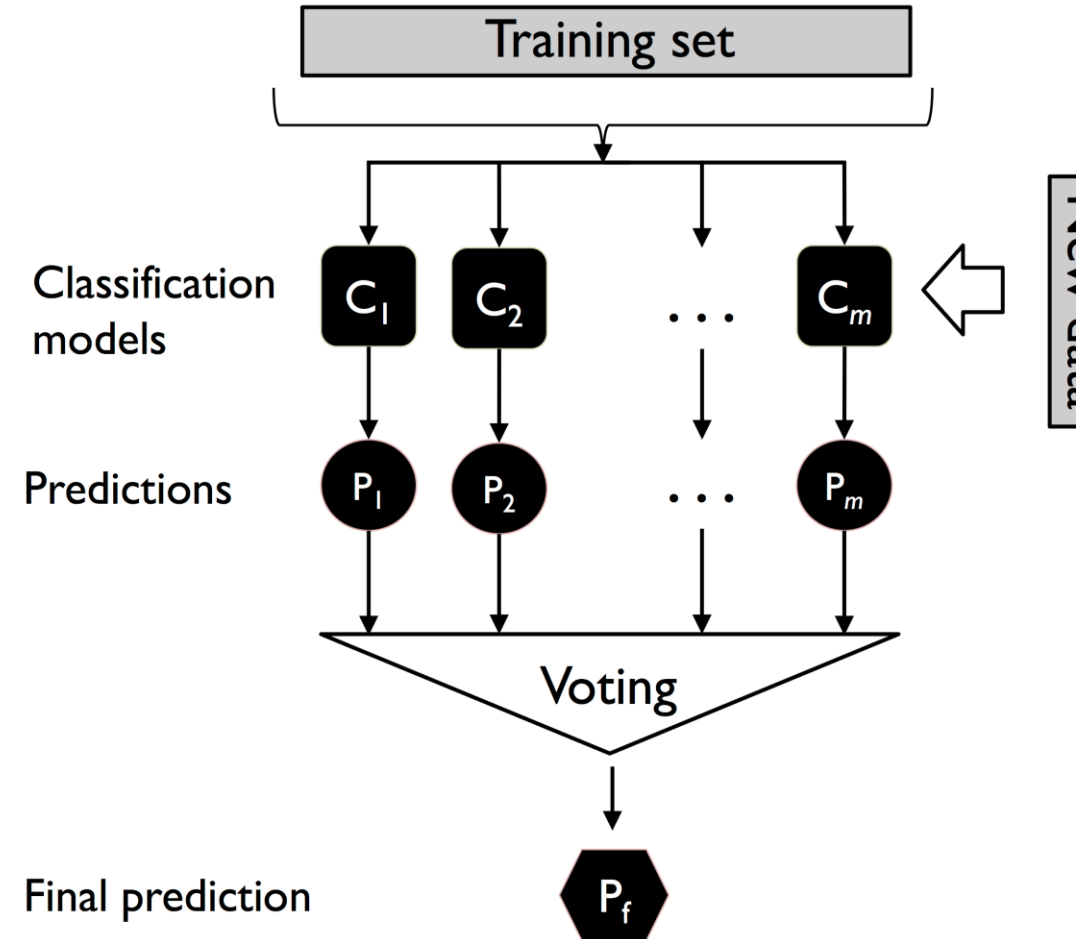


“TWO HEADS ARE BETTER THAN ONE”

Unknown

Ensemble Learning

The goal of Ensemble methods is to combine the prediction of different classifiers which will result in better generalization performance than each individual classifier alone.



Majority Vote Classifier

- Using majority voting, the class label $\hat{y}$ of m classifier C_j :

$$\hat{y} = \text{mode}\{C_1(x), C_2(x), \dots, C_m(x)\}$$

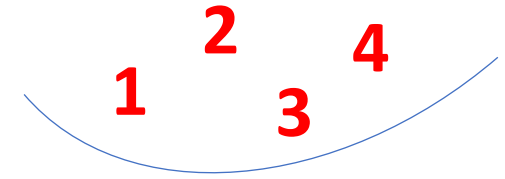
Weighted Majority Vote

- Weighted majority vote by associating a weight w_j with classifier C_j :

$$\hat{y} = \arg \max_i \sum_{j=1}^m w_j f_A (C_j(x) = i)$$

- w_j is a weight associated with a base classifier, C_j
- $\hat{y}$ is the predicated class label of the ensemble
- f_A is the characteristic function $[C_j(x) = i \in A]$
- A is the unique class label.

Some Definitions ...



- **Sampling with replacement**

- When a population element can be selected more than one time.

- **Sampling without replacement**

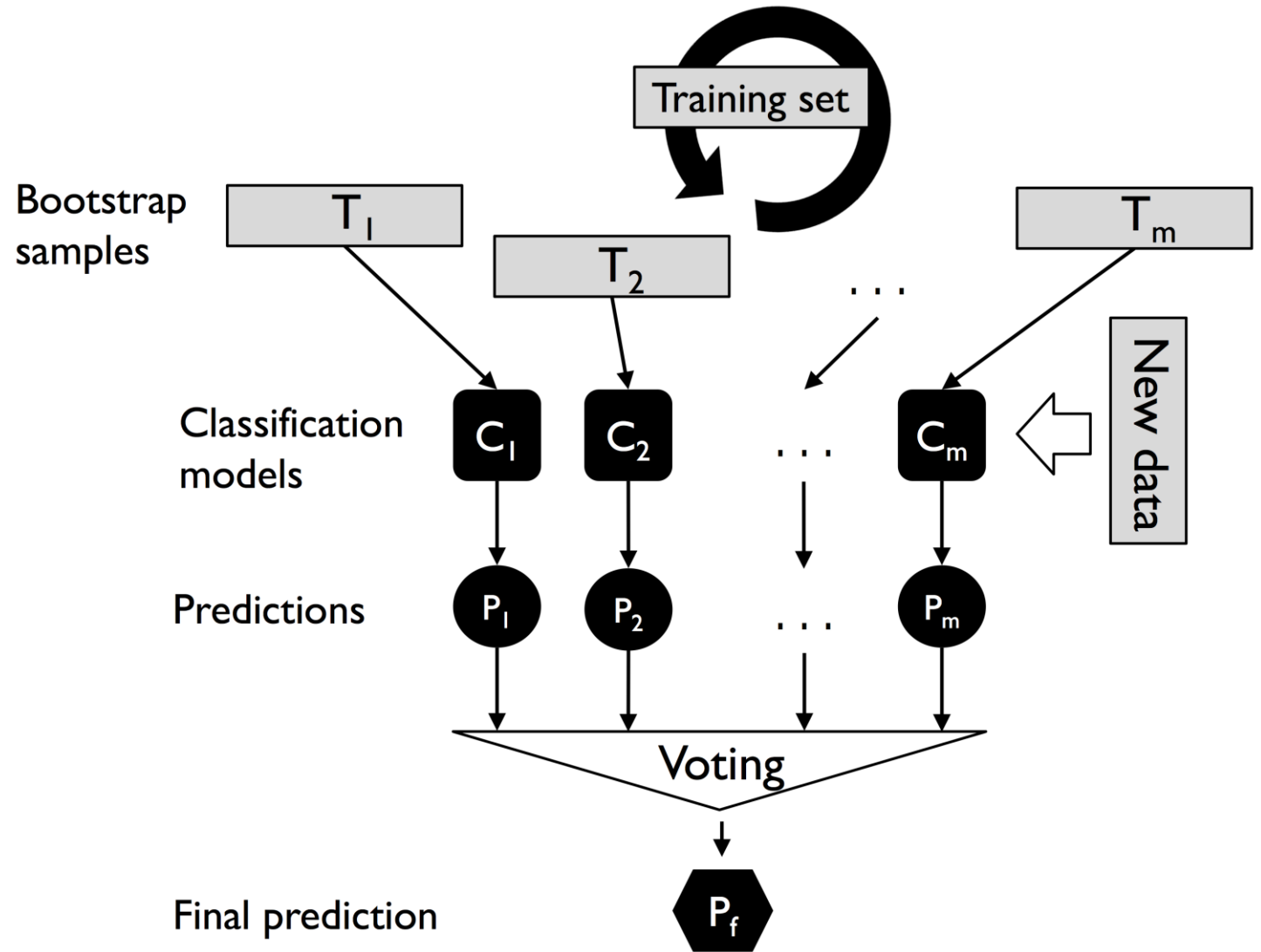
- When a population element can be selected only one time.

Model Ensembles

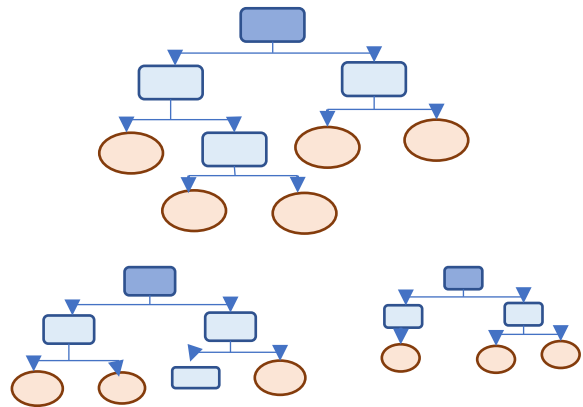
Two standard approaches:

- Bootstrap Aggregating (Bagging)
- Boosting

Bagging

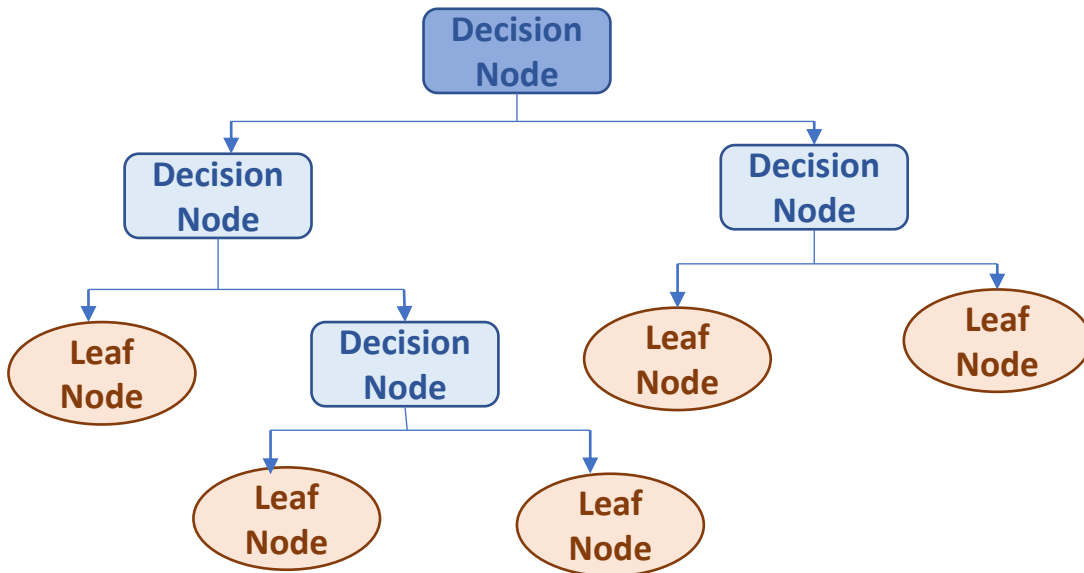


Random Forests



Random forest builds multiple decision trees and then merge their output to get a more accurate and stable prediction.

Recap: Decision Trees



- **Advantages to using decision trees:**

- Easy to interpret.
- Embedded feature selection.
- Can handle both numerical and categorical data.

- **Disadvantages of decision trees:**

- Changes in the population and variables distribution may cause wild swings in the model performance.
- Decision trees are prone to overfitting, especially when a tree is particularly deep.

Objective

- Build a Random Forest.
- Use a Random Forest for prediction/ classification.
- Calculate the accuracy of the Random Forest.
- Feature Importance using Random Forest.

Build Random Forest

Step 1: Draw a random “bootstrapped” sample dataset.

Step 2: Grow a decision tree using the bootstrapped dataset, at each node:

- Select random features to split on.

Step 3: Repeat step 1 & 2

Random Forests

Dataset

ID	F1	...	F20	Class
1	-	-	-	-
2	-	-	-	-
.	-	-	-	-
.	-	-	-	-
.	-	-	-	-
100	-	-	-	-

Bootstrapped Dataset_1

ID	F1	F2	F3	Class
2	-	-	-	-
2	-	-	-	-
.	-	-	-	-
.	-	-	-	-
.	-	-	-	-
4	-	-	-	-

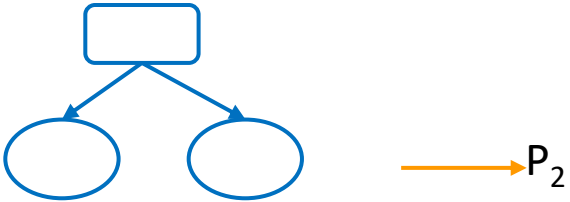
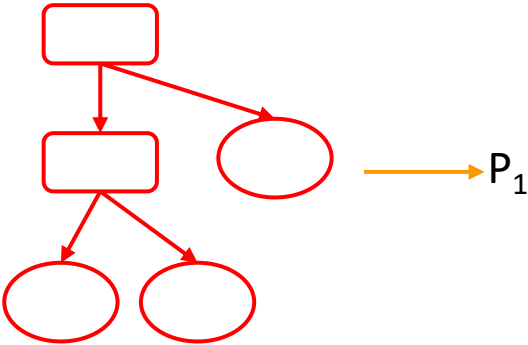
Bootstrapped Dataset_2

ID	F10	F20	...	F3	Class
55	-	-	-	-	-
33	-	-	-	-	-
.	-	-	-	-	-
.	-	-	-	-	-
.	-	-	-	-	-
90	-	-	-	-	-

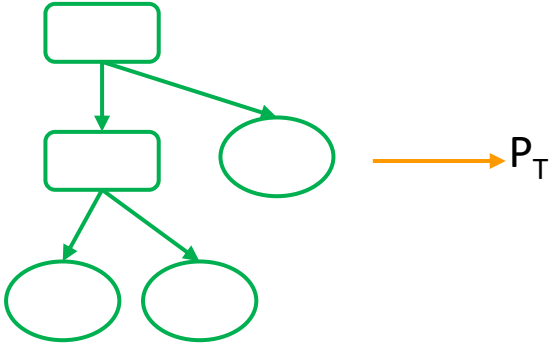
⋮

Bootstrapped Dataset_T

ID	F6	...	F8	Class
11	-	-	-	-
31	-	-	-	-
.	-	-	-	-
.	-	-	-	-
.	-	-	-	-
66	-	-	-	-



⋮



Random Forests

Dataset

ID	F1	F2	F3	Class
1	-	-	-	-
2	-	-	-	-
3	-	-	-	-
4	-	-	-	-

Bootstrapped Dataset_1

ID	F1	F2	F3	Class
2	-	-	-	-
2	-	-	-	-
4	-	-	-	-
4	-	-	-	-

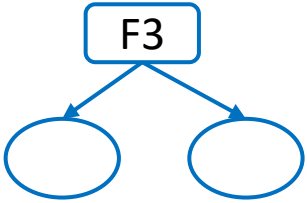
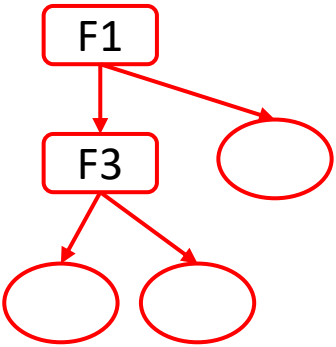
Bootstrapped Dataset_2

ID	F1	F2	F3	Class
1	-	-	-	-
3	-	-	-	-
3	-	-	-	-
4	-	-	-	-

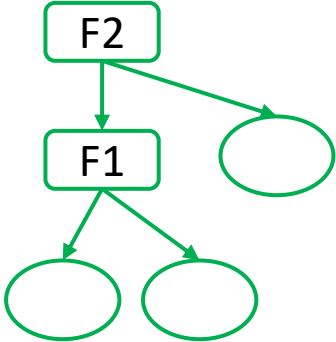
⋮

Bootstrapped Dataset_T

ID	F1	F2	F3	Class
1	-	-	-	-
1	-	-	-	-
2	-	-	-	-
3	-	-	-	-



⋮



Build Random Forest

Step 1: Draw a random “bootstrapped” sample dataset.

Step 2: Grow a decision tree using the bootstrapped dataset, at each node:

- Select features to split on.

Step 3: Repeat step 1 & 2

How a Random Forest Predicts

Aggregate the prediction by each tree to make final prediction by returning:

- **Majority voting :**
 - If we have Discrete target.
- **Mean:**
 - If we have Continuous target.

Random Forests

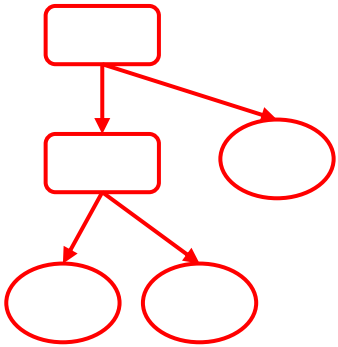
Dataset				
ID	F1	...	F20	Class
1	-	-	-	-
2	-	-	-	-
.	-	-	-	-
.	-	-	-	-
.	-	-	-	-
100	-	-	-	-

Bootstrapped Dataset_1				
ID	F1	F2	F3	Class
2	-	-	-	-
2	-	-	-	-
.	-	-	-	-
.	-	-	-	-
4	-	-	-	-

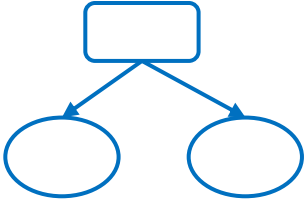
Bootstrapped Dataset_2					
ID	F10	F20	...	F3	Class
55	-	-		-	-
33	-	-		-	-
.	-	-		-	-
.					
.					
90	-	-		-	-

⋮

Bootstrapped Dataset_T				
ID	F6	...	F8	Class
11	-	-	-	-
31	-	-	-	-
.	-	-	-	-
.	-	-	-	-
66	-	-	-	-

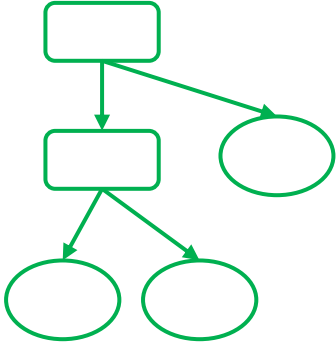


P₁



P₂

⋮



P_T

P_{final}

Regression:

$$\hat{y} = \frac{1}{T} \sum_{i=1}^T P_i$$

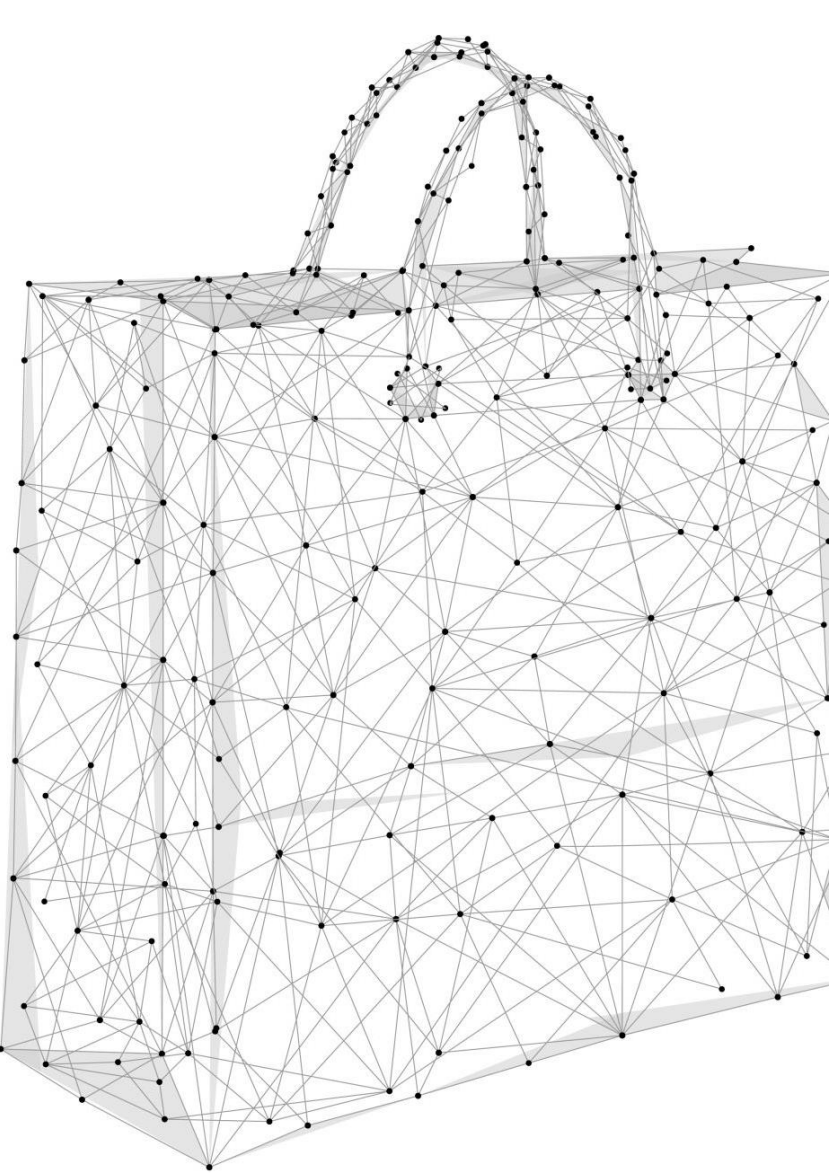
Classification:

$$\hat{y} = \text{mode}\{P_1, P_2, \dots, P_T\}$$

How many trees go in random forest?

It depends on many different variables such as

- the size of your training dataset,
 - number of attributes,
 - etc.
-
- A good idea is to start with fewer trees and move forward from there.



Out-of-Bag samples

- **Out-of-Bag samples (OOB)**
 - are the data records in the dataset that weren't included in the bootstrapped sample.
- **We can use OOB to:**
 - measure how accurate our random forest is
 - Out-of-Bag error: is the proportion of Out-of-Bag samples that were incorrectly classified.
 - Finding Missing value.
 - Feature selection
 - Attribute importance in the random forest.

Attribute Importance with OOB

Calculate the OOB error for the Random Forest.

This provides the “baseline” that will be used later for comparison.



A single attribute from the OOB dataset is tweaked or permuted at a time.

Selecting random value from a range of possible values for that attribute.

Attribute Importance with OOB

Calculate

Calculate the new OOB error for the attribute that was permuted.

Subtract

Subtract the new OOB from the baseline OOB we calculated at step 1.

Repeat

Repeat this process for every attribute.

Some Insights

- The random forest error rate depends on two things:
 - The *correlation* between any two trees in the forest.
 - Increasing the correlation increases the forest error rate.
 - The *strength* of each individual tree in the forest.
 - A tree with a low error rate is a strong classifier. Increasing the strength of the individual trees decreases the forest error rate.

When Do I use A Random Forest Vs. Decision Tree?



Random Forest

Higher accuracy.

Better generalization.



Decision Tree

Faster

Easier to interpret.

What about Multiple Classes?

- If we have more than 2 classes
 - the probability of each class is calculated
 - Select the highest as the algorithm final prediction.

Boosting

- Combine weak classifiers to obtain strong classifier.
 - Weak classifier
 - slightly better than random on training data
 - Resulting strong classifier
 - can eventually provide zero training error

Boosting Algorithm

Given: weak learner, a training data and number of iterations.

Procedure: run it multiple times on (reweighted) training data, then let learned classifiers vote.

On each iteration t :

- weight each training example by how incorrectly it was classified.
- Learn a weak hypothesis h_t
- A strength for this hypothesis α_t

Final classifier: $H(x) = \text{sign}(\sum_{t=1}^T \alpha_t h_t)$

Boosting Algorithm

- **INPUT:**

- Dataset D .
- Ensemble size T .
- Learning Algorithm A .

- **OUTPUT:**

- Weighted ensemble of models.

- **INITIALIZATION**

- Assign equal weights for all examples x_i in the dataset $w_{1i} = \frac{1}{|D|}$

Boosting Algorithm

- For $t = 1$ to T do
 - Run A algorithm on D with weights w_{ti} to produce a model h_t .
 - Calculate weighted error ϵ_t
 - If $\epsilon_t \geq 0.5$ then
 - Set $T \leftarrow t - 1$ and break
 - End If
 - $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$
 - Update weights

$$w_{t+1}(i) = \frac{w_t(i)e^{(-\alpha_t y_i h_t(x_i))}}{Z_t}$$

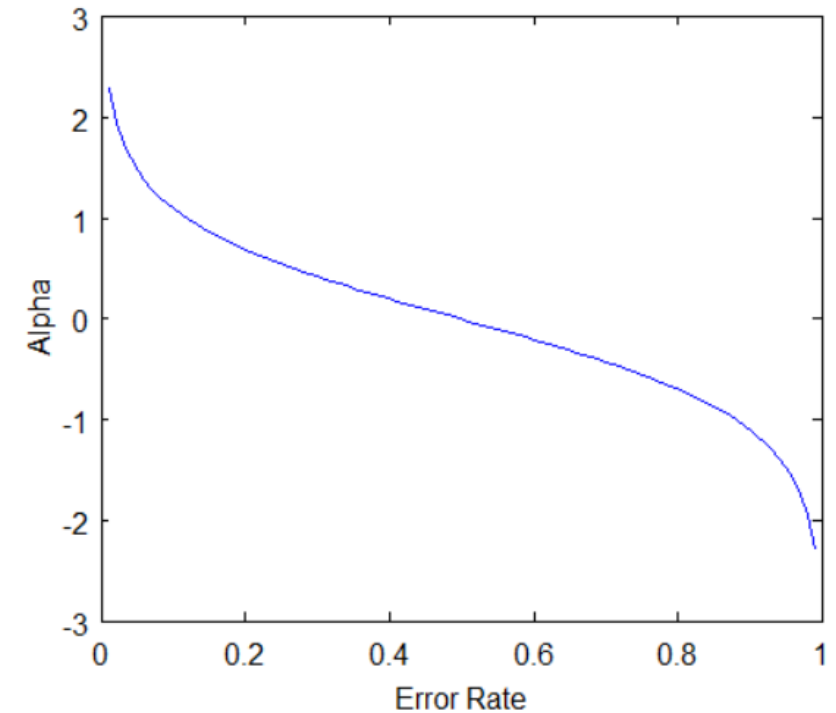
- End
- Return $H(x) = \text{sign}(\sum_{t=1}^T \alpha_t h_t)$

Classifier Output Weights (α_t)

$$\alpha_t = \frac{1}{2} \ln \frac{1 - \epsilon_t}{\epsilon_t},$$
$$0 < \epsilon_t \leq \frac{1}{2} - \gamma, \text{ for some } \gamma \in (0, \frac{1}{2})$$

After each classifier is trained, the classifier's weight is calculated based on its accuracy:

- More accurate classifiers are given more weight.
- A classifier with 50% accuracy is given a weight of zero.
- and a classifier with less than 50% accuracy is given negative weight.



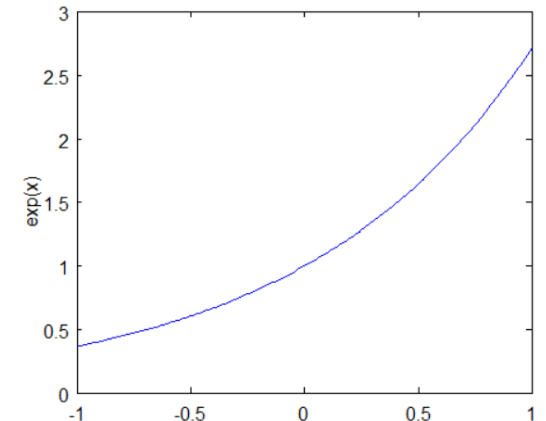
Boosting Algorithm

$w_{t+1}(i)$ increase the weight if the point is misclassified (if $y_i h_t(x_i) = -1$) and decrease it otherwise.

$$w_{t+1}(i) = \frac{w_t(i) e^{(-\alpha_t y_i h_t(x_i))}}{Z_t} = \frac{w_t(i)}{Z_t} \times \begin{cases} e^{-\alpha} & \text{if } h_t(x_i) = y_i \\ e^{\alpha} & \text{if } h_t(x_i) \neq y_i \end{cases}$$

where $Z_t = \sum_{i=1}^m w_t(i) e^{(-\alpha_t y_i h_t(x_i))}$

- Z_t is a normalization factor to ensure that the weights of all points should sum to 1.



Boosting Algorithm

The goal is to minimize weight error ϵ_t

$$\epsilon_t = \sum_{i=1}^m w_t(i) \delta(y_i \neq h_t(x_i)) \leq \prod_t Z_t$$

- $\epsilon_t = 0$ if h_t is perfectly classifies all weighted data points.
- $\epsilon_t = 1$ if h_t is perfectly wrong
- If we minimize $\prod_t Z_t$, we minimize our training error.
 - We can tighten this bound greedily, by choosing α_t and h_t on each iteration to minimize Z_t .

Boosting Algorithm

- choosing α_t on each iteration to minimize Z_t .

$$Z_t = \sum_{i:y_i = h_t(x_i)} w_t(i) e^{(-\alpha_t)} + \sum_{i:y_i \neq h_t(x_i)} w_t(i) e^{(\alpha_t)}$$

$$Z_t = e^{(-\alpha_t)} (1 - \epsilon_t) + e^{(\alpha_t)} \epsilon_t$$

- Taking the derivative with respect to α_t :

$$\frac{\partial Z_t}{\partial \alpha_t} = -e^{(-\alpha_t)} (1 - \epsilon_t) + e^{(\alpha_t)} \epsilon_t = 0 \implies e^{(2\alpha_t)} = \frac{1 - \epsilon_t}{\epsilon_t} \implies \alpha_t = \frac{1}{2} \ln \frac{1 - \epsilon_t}{\epsilon_t}$$

What are the main differences between Bagging and Boosting Models?

- How the algorithm works.
 - Parallel or sequentially?
- Base Model.
 - Strong learner or weak learner?
- Prediction.
 - Majority vote or weighted classifiers?
- Sampling Datasets
 - Bootstrapped datasets or weighted examples?

References

- Tom M. Mitchell, Machine learning, McGraw-Hill series in computer science, c1997, ISBN:0070428077
- Sebastian Raschka, Vahid Mirjalili, Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow, 2nd Edition, Packt Publishing, ISBN-13: 978-1787125933
- Chris Smith, Decision Trees and Random Forests, A visual Introduction for Beginners. Blue Windmill Media, 2017.

References

- Jiawei Han, Micheline Kamber, Jian Pei, Data Mining: Concepts and Techniques, Morgan Kaufmann; 3rd edition, ISBN-13: 978-9380931913.
- Kevin P. Murphy ,Machine Learning: A Probabilistic Perspective (Adaptive Computation and Machine Learning series), MIT Press, 1st Edition, ISBN-13: 978-0262018029, ISBN-10:0262018020
- Aoife D'Arcy, John D. Kelleher, Brian Mac Namee, Fundamentals of Machine Learning for Predictive Data Analytics: Algorithms, Worked Examples, and Case Studies, MIT Press , 1st Edition, ISBN-13: 978-0262029445.